

EquiLoan — Full Theory & Line-by-Line Code Walkthrough

For CS5403 PBL Review 1 — use this to explain any cell if asked in viva

This document explains **why** each stage of the notebook exists (the data science theory) and **what every line of code does** (the implementation). It follows the notebook top to bottom, section by section.

SECTION 0 — Imports & Setup (Cell 2)

Theory

Before any analysis, a data science pipeline needs three categories of tools:

1. **Data handling** — pandas (tabular data), numpy (numerical arrays)
2. **Visualization** — matplotlib (base plotting), seaborn (statistical plotting built on matplotlib)
3. **Modeling** — scikit-learn (Logistic Regression, Random Forest, splitting, scaling, metrics) and XGBoost (gradient boosting, installed separately since it isn't part of scikit-learn)

We also fix a `RANDOM_STATE = 42`. This is critical for **reproducibility** — every random process in ML (train/test split, Random Forest's bootstrap sampling, XGBoost's row/column subsampling) uses a pseudo-random number generator. Fixing the seed means anyone re-running the notebook gets identical results, which is required for your results to be verifiable in review.

Line-by-line

```
python

import pandas as pd
import numpy as np
```

Load pandas (DataFrame operations: `.read_csv`, `.groupby`, `.fillna`, etc.) and numpy (array math, used internally by pandas/sklearn and directly for things like `np.arange`).

```
python

import warnings
warnings.filterwarnings('ignore')
```

Suppresses non-fatal warning messages (e.g., sklearn convergence warnings, pandas deprecation notices) so the notebook output stays clean for a review audience. It does **not** hide errors — only warnings.

python

```
import matplotlib.pyplot as plt
import matplotlib.ticker as mtick
import seaborn as sns
%matplotlib inline
sns.set_theme(style='whitegrid', palette='muted')
plt.rcParams['figure.dpi'] = 110
```

- `matplotlib.pyplot` — the plotting interface (`plt.subplots`, `plt.show`).
- `matplotlib.ticker as mtick` — used later for `PercentFormatter` (turns 0.75 into "75%" on an axis).
- `seaborn` — higher-level statistical plots (`sns.boxplot`, `sns.heatmap`, `sns.countplot`).
- `%matplotlib inline` — Jupyter magic command that renders plots directly below the cell instead of in a separate window.
- `sns.set_theme(...)` — sets a global visual style (white grid background, muted color palette) so every chart in the notebook looks consistent.
- `plt.rcParams['figure.dpi'] = 110` — increases plot resolution/sharpness.

python

```
from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_score
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, roc_auc_score, confusion_matrix,
    classification_report, ConfusionMatrixDisplay, RocCurveDisplay
)
```

- `train_test_split` — splits data into training and test sets.
- `StratifiedKFold`, `cross_val_score` — imported for potential cross-validation (robustness check), not core to the required pipeline but available.
- `StandardScaler` — normalizes numeric features to mean 0, std 1 (needed because Logistic Regression is distance/gradient-based and sensitive to feature scale).
- `LabelEncoder` — imported as a utility, though this notebook mainly uses one-hot encoding (`pd.get_dummies`) instead.
- `LogisticRegression`, `RandomForestClassifier` — two of the three models.

- The metrics block imports every evaluation function used later: accuracy, precision, recall, F1, ROC-AUC, confusion matrix, and display helpers for plotting.

```
python

try:
    from xgboost import XGBClassifier
    XGB_AVAILABLE = True
except ImportError:
    import subprocess, sys
    subprocess.run([sys.executable, '-m', 'pip', 'install', 'xgboost', '-q'])
    from xgboost import XGBClassifier
    XGB_AVAILABLE = True
```

Defensive import: if XGBoost isn't already installed in the environment (common on fresh Colab/Kaggle kernels), the `except` block installs it silently via pip, then imports it. This makes the notebook portable — it won't crash on a machine where xgboost is missing.

```
python

RANDOM_STATE = 42
print('✅ All imports successful')
```

Sets the global seed used everywhere else in the notebook, and confirms setup succeeded.

SECTION 1 — Load & Inspect Data (Cells 4-9)

Theory

Before touching a dataset, a data scientist must **profile** it: how big is it, what are the column types, how much is missing, what does the target look like. Skipping this step is how people build models on data they don't actually understand. This maps to the **Data Understanding** phase of the CRISP-DM methodology.

Cell 4 — Load

```
python

df = pd.read_csv('loan_v2.csv')
print(f'Shape : {df.shape[0]:,} rows × {df.shape[1]} columns')
print(f'Memory : {df.memory_usage(deep=True).sum() / 1e6:.1f} MB')
df.head()
```

- `pd.read_csv` reads the CSV into a DataFrame `df`.

- `df.shape` returns `(rows, columns)`; the f-string prints it with thousands separators `(:,)`.
- `df.memory_usage(deep=True).sum()` computes actual memory footprint including string object overhead, converted to MB.
- `df.head()` displays the first 5 rows for a visual sanity check.

Cell 5 — Structure

python

```
df.info()
```

Prints column names, non-null counts, and dtypes for every column in one pass — this is how we first spot which columns have missing values (non-null count < total rows) and whether a numeric column was accidentally loaded as text (`object` dtype).

Cell 6 — Summary statistics

python

```
df.describe().T.style.background_gradient(cmap='Blues', subset=['mean', 'std', '5
```

- `.describe()` computes count, mean, std, min, quartiles, max for every numeric column.
- `.T` transposes so columns become rows (easier to read with many features).
- `.style.background_gradient(...)` is a pandas Styler call that color-shades the mean/std/median columns — purely cosmetic, helps visually spot columns with unusually large spread (a sign of the injected noise/outliers in `loan_v2.csv`).

Cell 7 — Missing values

python

```
missing = df.isnull().sum()
missing_pct = (missing / len(df) * 100).round(2)
missing_report = pd.DataFrame({'Missing Count': missing, 'Missing %': missing_pct})
missing_report = missing_report[missing_report['Missing Count'] > 0].sort_values('Missing Count')
print(f'Columns with missing values: {len(missing_report)}')
missing_report
```

- `df.isnull().sum()` counts NaNs per column.
- Converts to percentage of total rows.

- Combines both into a DataFrame, filters to only columns that actually have missing values, and sorts descending — so the worst offenders (`upfront_charges`, `interest_rate_spread`, `dtir1`, etc.) appear first. This directly informs the imputation strategy in Section 3.

Cell 8 — Cardinality

python

```
nunique = df.nunique().sort_values()
print('Unique values per column:')
print(nunique)
```

`df.nunique()` counts distinct values per column. This tells us which columns are binary/categorical (2-4 unique values, e.g. `gender`, `region`) versus continuous (thousands of unique values, e.g. `income`, `ltv`) — this distinction drives which columns get one-hot encoded versus scaled.

Cell 9 — Target distribution

python

```
TARGET = 'status'
print('Target column:', TARGET)
print(df[TARGET].value_counts())
print('Class balance (%):')
print((df[TARGET].value_counts(normalize=True) * 100).round(2))
```

Defines the single source-of-truth variable `TARGET` used everywhere downstream (so there's only one place to change if the column name were ever different). `value_counts()` shows raw counts of approved (0) vs rejected (1); `value_counts(normalize=True)` shows the same as proportions. Here it reveals **class imbalance** (~75% approved / ~25% rejected) — critical, because with imbalance, accuracy alone can be misleading (a model predicting "always approved" would score ~75% accuracy while being useless), which is why F1 and ROC-AUC are tracked throughout.

SECTION 2 — Exploratory Data Analysis / EDA (Cells 11-18)

Theory

EDA answers: *what does the data actually look like, and what problems does it have before I model it?* This notebook deliberately runs EDA **before** the preprocessing pipeline (not after, which was a bug in the original version) so that outlier/skew findings from EDA directly justify the IQR clipping step that follows.

Cell 11 — EDA working copy

python

```
DROP_COLS = [c for c in ['Unnamed: 0', 'id', 'year'] if c in df.columns]
df_eda = df.drop(columns=DROP_COLS).copy()

num_cols_eda = df_eda.select_dtypes(include=['int64', 'float64']).columns.tolist()
cat_cols_eda = df_eda.select_dtypes(include=['object']).columns.tolist()

for col in num_cols_eda:
    df_eda[col] = df_eda[col].fillna(df_eda[col].median())
for col in cat_cols_eda:
    df_eda[col] = df_eda[col].fillna(df_eda[col].mode()[0])

print(f'EDA dataframe shape: {df_eda.shape}')
print(f'Missing values remaining: {df_eda.isnull().sum().sum()}')
```

- `DROP_COLS` removes non-predictive identifier columns (`Unnamed: 0`, `id`, `year` — pure row indices, not signal).
- `.copy()` ensures `df_eda` is an independent DataFrame, so changes here never leak back into the original `df` (important: the raw `df` is reused later for the "before" baseline, so it must stay untouched).
- Splits columns into numeric vs categorical by dtype.
- **Median** imputation for numeric columns — median is robust to outliers/skew, unlike mean.
- **Mode** (most frequent value) imputation for categorical columns — you can't take a "mean" of a category.
- **Important distinction:** this imputed `df_eda` is used *only* for plotting. The real modelling pipeline (Section 3) redoes its own imputation independently on a fresh copy of `df`, so this step doesn't quietly leak into the model.

Cell 12 — Target distribution (visual)

python

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
vc = df_eda[TARGET].value_counts()
axes[0].bar(vc.index.astype(str), vc.values, color=['#2196F3', '#F44336'], edgec
...
axes[1].pie(vc.values, labels=[f'Status {i}' for i in vc.index], autopct='%1.1f
...
print('⚠ Class imbalance detected – accuracy alone is misleading. Use F1 + R
```

Two side-by-side plots: a bar chart (raw counts) and a pie chart (percentages) of approved vs rejected. `vc.index.astype(str)` converts the 0/1 labels to strings for clean axis labels. The final print statement is a **hard-coded reminder** of the class-imbalance implication established in Cell 9.

Cell 13 — Numeric distributions

```
python

num_features = [c for c in num_cols_eda if c != TARGET]
n_cols = 4
n_rows = (len(num_features) + n_cols - 1) // n_cols
fig, axes = plt.subplots(n_rows, n_cols, figsize=(20, n_rows * 4))
axes = axes.flatten()
for i, col in enumerate(num_features):
    axes[i].hist(df_eda[col].dropna(), bins=40, ...)
    skew = df_eda[col].skew()
    axes[i].text(..., f'Skew: {skew:.2f}', ..., color='red' if abs(skew) > 1 el
```

Builds a grid of histograms, one per numeric feature (excluding the target).

`(len(num_features) + n_cols - 1) // n_cols` is the standard "ceiling division" formula to compute how many rows of 4-across subplots are needed. Each subplot also computes **skewness** (`.skew()`) — a measure of distribution asymmetry — and colors it red if `|skew| > 1` (highly skewed, may need transformation) or green otherwise. This is how you'd visually spot the injected outliers in `loan_v2.csv` (e.g., `ltv`, `dtir1` will show extreme skew and long tails).

Cell 14 — Outlier detection (box plots)

```
python
```

```
iqr_cols = [c for c in ['income', 'loan_amount', 'property_value', 'rate_of_int
                    if c in df_eda.columns]

...
for ax, col in zip(axes, iqr_cols):
    ax.boxplot(df_eda[col].dropna(), vert=False, ...)
    Q1 = df_eda[col].quantile(0.25)
    Q3 = df_eda[col].quantile(0.75)
    IQR = Q3 - Q1
    outliers = ((df_eda[col] < Q1 - 1.5*IQR) | (df_eda[col] > Q3 + 1.5*IQR)).sum()
    ax.set_title(f'{col}\n({outliers:,} outliers)', ...)
```

Theory — the IQR method: the Interquartile Range ($IQR = Q3 - Q1$) measures the spread of the middle 50% of the data. Any value below ($Q1 - 1.5 \times IQR$) or above ($Q3 + 1.5 \times IQR$) is classified a statistical outlier — this is the standard Tukey's fence rule, widely used because it doesn't assume a normal distribution. This cell draws box plots (visual: box = IQR, whiskers = $1.5 \times IQR$ range, dots = outliers beyond that) for the six numeric columns most affected by the injected noise, and prints how many outliers each contains. This is the direct evidence that motivates clipping these exact six columns in Section 3.

Cell 15 — Correlation heatmap

python

```
num_only = df_eda.select_dtypes(include=['int64', 'float64'])
corr = num_only.corr()
mask = np.triu(np.ones_like(corr, dtype=bool))
sns.heatmap(corr, mask=mask, cmap='RdYlGn', center=0, annot=len(corr) <= 20, ...)
```

- `.corr()` computes the **Pearson correlation coefficient** (linear relationship strength, from -1 to +1) between every pair of numeric columns.
- `np.triu(...)` builds an upper-triangular boolean mask; passing it to `mask=` in the heatmap hides the upper triangle (since a correlation matrix is symmetric, showing both halves is redundant).
- `annot=len(corr) <= 20` only prints numeric values on the cells if there aren't too many columns (avoids visual clutter). This reveals which features move together (e.g., `loan_amount` and `property_value` likely correlate) and hints at multicollinearity, which matters more for Logistic Regression than for tree models.

Cell 16 — Categorical distributions

python

```
cat_features = [c for c in cat_cols_eda if c != TARGET]
...
for i, col in enumerate(cat_features):
    order = df_eda[col].value_counts().index
    sns.countplot(x=col, data=df_eda, order=order, ax=axes[i], hue=col, palette=
```

Bar chart of frequency counts for each categorical column (`gender`, `region`, `loan_type`, etc.), ordered from most to least frequent (`order=...`). `hue=col, legend=False` is used instead of a bare `palette=` argument because modern seaborn requires a `hue` mapping to apply a palette — passing `legend=False` prevents a redundant legend since the x-axis already labels each bar.

Cell 17 — SDG 10 fairness check (approval rate by group)

```
python

fairness_cols = [c for c in ['gender', 'loan_limit', 'loan_purpose', 'region',
for ax, col in zip(axes, fairness_cols):
    approval_rate = df_eda.groupby(col)[TARGET].mean().sort_values(ascending=Fa
...

```

Theory: `groupby(col)[TARGET].mean()` computes, for each category (e.g., each gender group), the *mean* of the 0/1 target — since the target is binary, the mean is the approval rate for that group. This is a first, purely descriptive look at potential bias, before any modelling — it directly serves the SDG 10 (Reduced Inequalities) angle of the project by showing whether approval rates already differ historically across demographic groups in the raw data.

Cell 18 — Financial features vs loan status

```
python

compare_cols = [c for c in ['income', 'loan_amount', 'property_value', 'credit_
for ax, col in zip(axes, compare_cols):
    sns.boxplot(x=df_eda[TARGET].astype(str), y=df_eda[col], ax=ax,
                hue=df_eda[TARGET].astype(str), legend=False,
                palette=['#EF9A9A', '#90CAF9'])

```

Box plots comparing the distribution of each key financial feature (income, loan amount, etc.) split by approval status — visually answers "do approved applicants have systematically higher income / lower loan amounts than rejected ones?"

`x=df_eda[TARGET].astype(str)` converts the numeric 0/1 target to strings, and the palette is

passed as a plain **list** (not a dict) matched positionally to the two groups — this avoids the earlier `ValueError` you hit, since a list has no keys to mismatch.

SECTION 3 — Preprocessing Pipeline (Cells 20–28)

Theory

This is the most important section for **methodological correctness**. Three data leakage traps are avoided here on purpose:

1. **Target leakage** — the target column must never be present when one-hot encoding features, or a "status" dummy column could sneak into X.
2. **Test-set leakage during imputation/scaling** — if you compute the median or the scaler's mean/std using the *entire* dataset (train+test combined) before splitting, information from the test set leaks into training, giving an artificially inflated (and invalid) accuracy. The fix: **split first, then scale using statistics learned only from the training set**.
3. **Order of operations** — outlier clipping must happen before the features are extracted into $\mathbf{X}$, not after, or the clipped values never make it into the model.

Cell 20 — Clean working copy

python

```
DROP_COLS = [c for c in ['Unnamed: 0', 'id', 'year'] if c in df.columns]
df_clean = df.drop(columns=DROP_COLS).copy()
print(f'Shape after dropping ID columns: {df_clean.shape}')
```

Starts a fresh, independent copy from the *original* (`df`) (not `df_eda`), dropping the same identifier columns as before. `df_clean` is the single source of truth from here through model training.

Cell 21 — Drop rows with null target

python

```
before = len(df_clean)
df_clean = df_clean[df_clean[TARGET].notna()].copy()
after = len(df_clean)
print(f'Rows removed (null target): {before - after:,}')
```

You cannot train a supervised model on a row with no label. `.notna()` builds a boolean mask keeping only rows where `status` is not missing. Since your dataset had 0 missing targets, this is a safety guard rather than something that visibly changes the data here — but it's essential defensive coding for any dataset.

Cell 22 — Drop duplicates

python

```
before = len(df_clean)
df_clean = df_clean.drop_duplicates()
after = len(df_clean)
```

Removes exact duplicate rows, which would otherwise bias the model toward whatever pattern is duplicated and also risk the same row appearing in both train and test splits.

Cell 23 — Impute missing values

python

```
num_cols = df_clean.select_dtypes(include=['int64', 'float64']).columns.tolist()
cat_cols = df_clean.select_dtypes(include=['object']).columns.tolist()
num_cols = [c for c in num_cols if c != TARGET]
cat_cols = [c for c in cat_cols if c != TARGET]
for col in num_cols:
    df_clean[col] = df_clean[col].fillna(df_clean[col].median())
for col in cat_cols:
    df_clean[col] = df_clean[col].fillna(df_clean[col].mode()[0])
```

Same median/mode strategy as the EDA copy, but now applied to the real modelling DataFrame. The target is explicitly excluded from both lists so it's never accidentally "imputed."

Cell 24 — IQR clipping

python

```
iqr_cols = [c for c in ['income', 'loan_amount', 'property_value', 'rate_of_int
                if c in df_clean.columns]
clip_log = []
for col in iqr_cols:
    Q1 = df_clean[col].quantile(0.25)
    Q3 = df_clean[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    n_clipped = ((df_clean[col] < lower) | (df_clean[col] > upper)).sum()
    df_clean[col] = df_clean[col].clip(lower, upper)
    clip_log.append({...})
```

Theory — clipping vs. removing: instead of *deleting* outlier rows (which would shrink the dataset and could remove real, valid extreme cases), we **clip** ("winsorize") them — any value beyond the fences is capped at the fence value itself. This keeps the row (and its other feature values, and its label) in the dataset while neutralizing the distortion an extreme value would cause, especially for scale-sensitive models like Logistic Regression.

`df[col].clip(lower, upper)` is pandas' built-in method that does exactly this in one call.

This is the step that does the real work on the injected noise in `loan_v2.csv` — your earlier run showed 9,590 values clipped in `ltv` and 9,179 in `dtir1`.

Cell 25 — Extract X and y

python

```
X = df_clean.drop(columns=[TARGET]).copy()
y = df_clean[TARGET].copy()
assert TARGET not in X.columns, f'BUG: {TARGET} is still in X!'
```

Separates features (`(X)`) from the label (`(y)`). The `assert` statement is a **defensive check** — if it ever fails, the notebook stops immediately with a clear error instead of silently training a model that can see its own answer (perfect target leakage).

Cell 26 — One-hot encoding

python

```
X_encoded = pd.get_dummies(X, drop_first=True)
X_encoded.columns = (
    X_encoded.columns
    .str.replace(r'[^a-zA-Z0-9_]', '_', regex=True)
    .str.strip('_')
)
status_leak = [c for c in X_encoded.columns if 'status' in c.lower()]
if status_leak:
    X_encoded = X_encoded.drop(columns=status_leak)
```

Theory: ML models need numeric input. `pd.get_dummies` converts each categorical column into multiple binary (0/1) columns, one per category. `drop_first=True` drops one category per feature to avoid the "dummy variable trap" (perfect multicollinearity, where one dummy column is fully predictable from the others). The column-name cleanup (regex replacing special characters with underscores) exists because XGBoost historically rejects feature names containing brackets, commas, or other special characters. The final `status_leak` check is a second, redundant safety net (on top of the Cell 25 assert) — good practice in a project focused on catching bugs like this.

Cell 27 — Train/test split

python

```
X_train_raw, X_test_raw, y_train, y_test = train_test_split(
    X_encoded, y, test_size=0.2, random_state=RANDOM_STATE, stratify=y
)
```

80/20 split. `stratify=y` ensures the train and test sets preserve the same class balance (~75/25) as the full dataset — without this, a random split could accidentally put too many/few rejected-loan examples in the test set, skewing evaluation.

`random_state=RANDOM_STATE` makes the split reproducible.

Cell 28 — Scaling (fit on train only)

python

```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train_raw)
X_test = scaler.transform(X_test_raw)
```

Theory — why fit only on train: `StandardScaler` transforms each feature to mean 0, std 1 using the formula $(x - \text{mean}) / \text{std}$. `.fit_transform()` on the training set *learns* the mean/std from training data and applies them. `.transform()` (not `fit_transform`) on the test set *reuses those same learned statistics* rather than recomputing new ones from the test data. This is the textbook-correct way to prevent test-set information from leaking into preprocessing — if you scaled before splitting, the mean/std would include test rows, contaminating the "unseen data" evaluation.

SECTION 4 — Baseline Models / "Before" (Cells 30-32)

Theory

To claim your preprocessing pipeline "improves" the model, you need a genuine control group — a version of the pipeline with the advanced steps (IQR clipping, scaling) removed, but everything else identical (same split, same imputation, same models). That's what `df_base` represents.

Cell 30 — Minimal-preprocessing baseline

python

```

df_base = df.drop(columns=DROP_COLS_BASE).copy()
df_base = df_base[df_base[TARGET].notna()].copy()
for col in df_base.select_dtypes(include=['int64', 'float64']).columns:
    if col != TARGET:
        df_base[col] = df_base[col].fillna(df_base[col].median())
for col in df_base.select_dtypes(include=['object']).columns:
    if col != TARGET:
        df_base[col] = df_base[col].fillna(df_base[col].mode()[0])
X_base = df_base.drop(columns=[TARGET])
y_base = df_base[TARGET]
X_base = pd.get_dummies(X_base, drop_first=True)
X_base.columns = X_base.columns.str.replace(r'^a-zA-Z0-9_', '_', regex=True)
status_leak_base = [c for c in X_base.columns if 'status' in c.lower()]
if status_leak_base:
    X_base = X_base.drop(columns=status_leak_base)
X_train_base, X_test_base, y_train_base, y_test_base = train_test_split(
    X_base, y_base, test_size=0.2, random_state=RANDOM_STATE, stratify=y_base
)

```

Rebuilds the exact same imputation + encoding + split logic as Section 3, starting again from the raw `df` — but **deliberately skips** the IQR clipping (Cell 24 equivalent) and skips scaling entirely (`X_train_base`/`X_test_base` are used raw, unscaled). This is the "before advanced preprocessing" arm of the experiment.

Cell 31 — Evaluation function + model factory

python

```
def evaluate_model(model, X_tr, X_te, y_tr, y_te, model_name='Model'):
    model.fit(X_tr, y_tr)
    y_pred = model.predict(X_te)
    y_prob = model.predict_proba(X_te)[: , 1]
    return {
        'Accuracy' : round(accuracy_score(y_te, y_pred), 4),
        'Precision': round(precision_score(y_te, y_pred, zero_division=0), 4),
        'Recall'    : round(recall_score(y_te, y_pred, zero_division=0), 4),
        'F1 Score'  : round(f1_score(y_te, y_pred, zero_division=0), 4),
        'ROC AUC'   : round(roc_auc_score(y_te, y_prob), 4)
    }

def get_fresh_models():
    return {
        'Logistic Regression': LogisticRegression(max_iter=1000, random_state=R
        'Random Forest'      : RandomForestClassifier(n_estimators=100, random_
        'XGBoost'            : XGBClassifier(n_estimators=100, random_state=RAN
    }
```

Theory of each metric:

- **Accuracy** — fraction of correct predictions overall. Misleading under class imbalance.
- **Precision** — of everyone the model predicted "rejected," how many actually were? (Cost of false alarms.)
- **Recall** — of everyone who was actually "rejected," how many did the model catch? (Cost of missed cases.)
- **F1 Score** — harmonic mean of precision and recall; a single balanced number, especially useful under imbalance.
- **ROC-AUC** — the model's ability to *rank* positive cases above negative cases across all possible decision thresholds, not just the default 0.5 cutoff. `y_prob = model.predict_proba(X_te)[: , 1]` extracts the predicted *probability* of class 1 (rejected), which ROC-AUC needs (not just the hard 0/1 prediction).

`get_fresh_models()` is a **factory function** — it returns brand-new, untrained model instances every time it's called. This matters because reusing an already-`.fit()`-ted model object across the "before" and "after" runs would mean the second run's `.fit()` call trains *on top of* the first (or, for models that don't support incremental fitting like sklearn's, silently overwrites internal state in ways that make comparisons unreliable) — fresh instances guarantee each run is a clean, independent experiment.

Cell 32 — Run baseline

```
python

before_results = {}
baseline_models = get_fresh_models()
for name, model in baseline_models.items():
    before_results[name] = evaluate_model(model, X_train_base, X_test_base, y_train_base)
before_df = pd.DataFrame(before_results).T
```

Loops through all 3 fresh models, trains and evaluates each on the *unclipped, unscaled* baseline data, collects the metric dicts into `before_results`, then converts to a DataFrame (`.T` transposes so model names become row labels).

SECTION 5 — Full Pipeline Models / "After" (Cell 34)

```
python

after_results = {}
full_models = get_fresh_models()
for name, model in full_models.items():
    after_results[name] = evaluate_model(model, X_train, X_test, y_train, y_test)
after_df = pd.DataFrame(after_results).T
```

Identical loop structure to Cell 32, but trained on `X_train`/`X_test` — the IQR-clipped, scaled data from Section 3. Note `full_models` (not `baseline_models`) is kept as a separate dict of *already-fitted* model objects, deliberately reused later in Cells 42-43 for the confusion matrix / ROC curve plots, so those models don't need to be retrained from scratch.

SECTION 6 — Before vs After Comparison (Cells 36-39)

Cell 36 — Comparison table

```
python
```

```
comparison = pd.DataFrame({
    'Before Accuracy' : before_df['Accuracy'],
    'After Accuracy'  : after_df['Accuracy'],
    ...
})
comparison['Accuracy Δ'] = (after_df['Accuracy'] - before_df['Accuracy']).round(2)
...
comparison.style.map(
    lambda v: 'color: green; font-weight: bold' if isinstance(v, float) and v > 0
    else ('color: red' if isinstance(v, float) and v < 0 else ''),
    subset=['Accuracy Δ', 'F1 Δ', 'ROC AUC Δ']
)
```

Builds a single side-by-side table of before/after metrics plus the delta (Δ , After – Before) for each. `.style.map(...)` applies conditional formatting cell-by-cell: positive deltas render green/bold (improvement), negative render red (regression) — a quick visual scan for your reviewers.

Cell 38 — Grouped bar chart

python

```
for ax, metric in zip(axes, metrics_to_plot):
    b_vals = before_df[metric].values
    a_vals = after_df[metric].values
    ax.bar(x - width/2, b_vals, width, label='Before', ...)
    ax.bar(x + width/2, a_vals, width, label='After', ...)
```

For each of Accuracy/F1/ROC-AUC, draws paired bars (before in light blue, after in dark blue) per model, offset by $\pm \text{width}/2$ so they sit side by side rather than overlapping — the standard "grouped bar chart" pattern for before/after comparisons.

Cell 39 — Best model

python

```
best_model_name = after_df['F1 Score'].idxmax()
```

`.idxmax()` returns the row label (model name) with the highest F1 score in the "after" results — F1 is chosen as the selection criterion (not raw accuracy) precisely because of the class imbalance noted in Cell 9.

SECTION 7 — Deep Dive: Best Model Analysis (Cells 41-43)

Cell 41 — Confusion matrix + ROC curve

python

```
best_model = get_fresh_models()[best_model_name]
best_model.fit(X_train, y_train)
y_pred_best = best_model.predict(X_test)
y_prob_best = best_model.predict_proba(X_test)[:, 1]
cm = confusion_matrix(y_test, y_pred_best)
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot(...)
RocCurveDisplay.from_predictions(y_test, y_prob_best, ...)
print(classification_report(y_test, y_pred_best))
```

Retrains a **fresh** instance of the winning model (same reasoning as before — never reuse a fitted object across experiments) on the full "after" pipeline data. The **confusion matrix** is a 2×2 grid of True Negatives / False Positives / False Negatives / True Positives — the raw counts behind precision/recall/F1. The **ROC curve** plots True Positive Rate vs False Positive Rate across every possible probability threshold; the diagonal dashed line represents random guessing (AUC = 0.5), so the further the curve bows toward the top-left corner, the better. `classification_report` prints precision/recall/F1 per class plus macro/weighted averages in one readable block.

Cell 42 — Compare all 3 ROC curves

python

```
for (name, model), color in zip(full_models.items(), colors):
    y_prob_m = model.predict_proba(X_test)[:, 1]
    RocCurveDisplay.from_predictions(y_test, y_prob_m, ax=plt.gca(), name=name,
```

Overlays all three models' ROC curves on one plot for direct visual comparison. Note this reuses `full_models` — the dictionary of *already-fitted* models from Cell 34 — so no retraining happens here, just re-running `.predict_proba()`.

Cell 43 — Feature importance

python

```
importance_model_name = 'Random Forest' if 'Random Forest' in full_models else
importance_model = full_models[importance_model_name]
if hasattr(importance_model, 'feature_importances_'):
    feat_imp = pd.Series(importance_model.feature_importances_, index=X_encoded
```

Tree-based models (Random Forest, XGBoost) expose a `.feature_importances_` attribute — for each feature, how much it reduced impurity (Gini/entropy) across all the trees' splits, summed and normalized. Logistic Regression doesn't have this attribute (it has coefficients instead, which mean something different), so the code specifically picks a tree model for this analysis via `hasattr(...)`. Sorting descending shows which features (e.g., `credit_score`, `rate_of_interest`) drive the model's decisions most — and lets you cross-check whether a fairness-sensitive feature like `senior_age` ranks unexpectedly high, which your run flagged for review.

SECTION 8 — Fairness Audit / SDG 10 (Cell 45)

Theory

This is the project's core novelty beyond the base paper. Two standard fairness metrics from the algorithmic fairness literature:

- **Statistical Parity Difference (SPD)** = (approval rate of the highest-approved group) – (approval rate of the lowest-approved group). Ideal value: **0** (no gap between groups).
- **Disparate Impact Ratio (DIR)** = (approval rate of lowest-approved group) ÷ (approval rate of highest-approved group). Ideal value: **1.0**. A DIR **below 0.8** is the widely cited "**four-fifths rule**" from U.S. EEOC employment discrimination guidelines, adapted here to loan approvals — if the most-disadvantaged group's approval rate is less than 80% of the most-advantaged group's, that's flagged as evidence of disparate impact.

python

```

test_idx = X_test_raw.index
df_test_fair = df_clean.loc[test_idx].copy()
df_test_fair['predicted'] = y_pred_best
df_test_fair['actual'] = y_test.values

fairness_audit_cols = [c for c in ['gender', 'region', 'loan_limit', 'loan_purp
for col in fairness_audit_cols:
    group_rates = df_test_fair.groupby(col)['predicted'].mean()
    if len(group_rates) >= 2:
        max_rate = group_rates.max()
        min_rate = group_rates.min()
        spd = round(max_rate - min_rate, 4)
        dir_ = round(min_rate / max_rate, 4) if max_rate > 0 else None
        fairness_results.append({... , 'Biased?': '⚠️ YES' if (dir_ is not None

```

- `test_idx = X_test_raw.index` recovers the original row indices belonging to the test set (since `X_train`/`X_test` after scaling are plain numpy arrays with no index, but `X_test_raw` — the pre-scaling DataFrame — still has them).
- `df_clean.loc[test_idx]` pulls the original, human-readable columns (`gender`, `region`, etc. — pre-encoding) for exactly those test rows, since one-hot encoding earlier destroyed those readable category labels in `X_encoded`.
- Attaches the model's `predicted` labels and the `actual` ground truth back onto this readable DataFrame.
- For each sensitive attribute, `groupby(col)['predicted'].mean()` computes what fraction of *predicted* approvals each demographic group received — the model's own decision pattern, not the historical/raw data pattern from Cell 17. Comparing the two (historical bias in Cell 17 vs. model bias here) is a meaningful before/after fairness comparison for your report.
- SPD and DIR are computed exactly per the formulas above, and the `Biased?` flag applies the four-fifths rule automatically.

Your last run found `gender`, `loan_limit`, `loan_purpose`, and `age` all flagged (DIR 0.64–0.71), with `region` passing (DIR 0.82) — this is a genuine, reportable SDG 10 finding: the model, even though it never sees the target directly during training beyond learning from historical outcomes, reproduces disparate approval patterns across demographic groups.

SECTION 9 — Final Summary (Cells 47–49)

Cell 47 — Text summary

python

```
print(before_df.to_string())
print(after_df.to_string())
delta = after_df - before_df
print(delta.to_string())
print(f'🏆 Best model (by F1): {best_model_name}')
```

Prints a plain-text, presentation-ready summary of everything computed so far: full before/after metric tables, the DataFrame-level delta (subtracting two DataFrames element-wise, aligned by row/column labels), and the winning model — useful to paste directly into your report or slides.

Cell 48 — Save results

python

```
before_df.to_csv('Before_Preprocessing_Results.csv')
after_df.to_csv('After_Preprocessing_Results.csv')
comparison.to_csv('Before_After_Comparison.csv')
```

Persists the three key result tables to disk as CSVs — these are the artifacts you'd attach to your Review 1 report or upload alongside the notebook as evidence.

Quick-reference: what each PBL deliverable maps to

Review requirement	Notebook section
Data understanding	Section 1
EDA (histogram, boxplot, countplot, heatmap)	Section 2
Missing value handling (median/mode)	Cells 11, 23
Feature engineering (<code>high_interest_rate</code> , <code>senior_age</code>)	Present in source CSV, visible from Cell 5 onward
Preprocessing (IQR clipping, one-hot encoding, scaling)	Section 3
Multi-model comparison (LogReg / RF / XGBoost)	Sections 4-6
Evaluation metrics (Accuracy, Precision, Recall, F1, ROC-AUC, Confusion Matrix)	Cell 31 (function), Cells 36-42 (results)
Demographic fairness audit (SPD, DIR) — SDG 10	Cell 17 (raw bias), Cell 45 (model bias)
Reproducibility / leakage-free pipeline	<code>RANDOM_STATE</code> , split-before-scale (Cells 27-28), target-leakage asserts (Cells 25-26)

Use this document as your speaking notes for viva — for any cell number, you now have both the "why" (theory) and the "what" (line-by-line) ready to explain.